

Chapter 10. Domain-specific languages using lambdas

This chapter covers

- What domain-specific languages (DSLs) and their forms are
- The pros and cons of adding a DSL to your API
- The alternatives available on the JVM to a plain Java-based DSL
- Learning from the DSLs present in modern Java interfaces and classes
- Patterns and techniques to implement effective Java-based DSLs
- How commonly used Java libraries and tools use these patterns

Developers often forget that a programming language is first of all a language. The main purpose of any language is to convey a message in the clearest, most understandable way. Perhaps the most important characteristic of well-written software is clear communication of its intentions—or, as famous computer scientist Harold Abelson stated, “Programs must be written for people to read and only incidentally for machines to execute.”

Readability and understandability are even more important in the parts of the software that are intended to model the core business of your application. Writing code that can be shared and understood by both the development team and the domain experts is helpful for productivity. Domain experts can be involved in the software development process and verify the correctness of the software from a business point of view. As a result, bugs and misunderstandings can be caught early on.

To achieve this result, it's common to express the application's business logic through a domain-specific language (DSL). A *DSL* is a small, usually non-general-purpose programming language explicitly tailored for a specific domain. The DSL uses the terminology characteristic of that domain. You may be familiar with Maven and Ant, for example. You can see them as DSLs for expressing build processes. You're also familiar with HTML,

Other formats available



Audiobook



define the structure of a web page.

and excessive verbosity, Java has never

g a compact DSL that's also suitable to be

read by non-technical people. Now that Java supports lambda expressions, however, you have new tools in your toolbox! In fact, you learned in [chapter 3](#) that lambda expressions help reduce code verbosity and improve the signal/noise ratio of your programs.

Think about a database implemented in Java. Deep down in the database, there's likely to be lots of elaborate code determining where on disk to store a given record, constructing indexes for tables, and dealing with concurrent transactions. This database is likely to be programmed by relatively expert programmers. Suppose that now you want to program a query similar to those we explored in [chapters 4](#) and [5](#): “Find all menu entries on a given menu that have fewer than 400 calories.”

Historically, such expert programmers might have quickly written low-level code in this style and thought that the task was easy:

```
while (block != null) {
    read(block, buffer)
    for (every record in buffer) {
        if (record.calorie < 400) {
            System.out.println (record.name);
        }
    }
    block = buffer.next();
}
```

This solution has two main problems: it's hard for a less-experienced programmer to create (it may need subtle details of locking, I/O, or disc allocation), and more important, it deals in system-level concepts, not application-level concepts.

A new-joining user-facing programmer might say, “Why can't you provide me an SQL interface so I can write `SELECT name FROM menu WHERE calorie < 400`, where `menu` holds the restaurant menu expressed as an SQL table? Now I can program far more effectively than all this system-level nonsense!” It's hard to argue with this statement! In essence the programmer has asked for a DSL to interact with the database instead of writing pure Java code. Technically, this type of DSL is called *external* because it expects the database to have an API that can parse and evaluate SQL expressions written in text. You learn more about the distinction between external and internal DSL later in this chapter.

But if you think back to [chapters 4](#) and [5](#), you notice that this code could also be written more concisely in Java using the `Stream` API, such as the

Other formats available



```
.filter(d -> d.getCalories() < 400)
.map(Dish::getName)
.forEach(System.out::println)
```

This use of chaining methods, which is so characteristic of the `Stream` API, is often called *fluent style* in that it's easy to understand quickly, in contrast to complex control flow in Java loops.

This style effectively captures a DSL. In this case, this DSL isn't external, but internal. In an *internal* DSL, the application-level primitives are exposed as Java methods to use on one or more class types that represent the database, in contrast to the non-Java syntax for primitives in an external DSL, such as `SELECT FROM` in the SQL discussion above.

In essence, designing a DSL consists of deciding what operations the application-level programmer needs to manipulate (carefully avoiding any unnecessary pollution caused by system-level concepts) and providing these operations to the programmer.

For an internal DSL, this process means exposing appropriate classes and methods so that code can be written fluently. An external DSL takes more effort; you must not only design the DSL syntax, but also implement a parser and evaluator for the DSL. If you get the design right, however, perhaps lower-skilled programmers can write code quickly and effectively (thus making the money that keeps your company in business) without having to program directly within your beautiful (but hard for non-experts to understand) system-level code!

In this chapter, you learn what a DSL is through several examples and use cases; you learn when you should consider implementing one and what the benefits are. Then you explore some of the small DSLs that were introduced in the Java 8 API. You also learn how you could employ the same patterns to create your own DSLs. Finally, you investigate how some widely used Java libraries and frameworks have adopted these techniques to offer their functionalities through a set of DSLs, making their APIs more accessible and easy to use.

10.1. A specific language for your domain

A DSL is a custom-built language designed to solve a problem for a specific business domain. You may be developing a software application for accounting, for example. Your business domain includes concepts such as bank statements and operations such as reconciling. You could create a

Other formats available



business domain.

× ns in that domain. In Java, you need to
nd methods to represent that domain. In a
API created to interface with a specific

A DSL isn't a general-purpose programming language; it restricts the operations and vocabulary available to a specific domain, which means that

you have less to worry about and can invest more attention in solving the business problem at hand. Your DSL should allow its users to deal only with the complexities of that domain. Other lower-level implementation details should be hidden – just like making lower-level implementation-detail methods of a class private. This results in a user-friendly DSL.

What isn't a DSL? A DSL isn't plain English. It's also not a language that lets domain experts implement low-level business logic. Two reasons should drive you toward the development of a DSL:

- ***Communication is king.*** Your code should clearly communicate its intentions and be understandable even by a non-programmer. This way, this person can contribute to validating whether the code matches the business requirements.
- ***Code is written once but read many times.*** Readability is vital for maintainability. In other words, you should always code in a way that your colleagues thank you for rather than hate you for!

A well-designed DSL offers many benefits. Nonetheless, developing and using a bespoke DSL has pros and cons. In [section 10.1.1](#), we explore the pros and cons in more detail so that you can decide when a DSL is appropriate (or not) for a particular scenario.

10.1.1. Pros and cons of DSLs

DSLs, like other technologies and solutions in software development, aren't silver bullets. Using a DSL to work with your domain can be both an asset and a liability. A DSL can be an asset because it raises the level of abstraction with which you can clarify the business intention of the code and makes the code more readable. But it can also be a liability because the implementation of the DSL is code in its own right that needs to be tested and maintained. For this reason, it's useful to investigate the benefits and costs of DSLs so that you can evaluate whether adding one to your project will result in a positive return of investment.

DSLs offer the following benefits:

- ***Conciseness***— An API that conveniently encapsulates the business logic allows you to avoid repetition, resulting in code that's less

Other formats available



Audiobook



that belong to the vocabulary of the domain. This way, the business logic is understandable even by domain non-experts. Domain knowledge can be shared across a

wider range of members of the organization.

- ***Maintainability***— Code written against a well-designed DSL is easier to maintain and modify. Maintainability is especially important for

business-related code, which is the part of the application that may change most frequently.

- **Higher level of abstraction**— The operations available in a DSL work at the same level of abstraction as the domain, thus hiding the details that aren't strictly related to the domain's problems.
- **Focus**— Having a language designed for the sole purpose of expressing the rules of the business domain helps programmers stay focused on that specific part of the code. The result is increased productivity.
- **Separation of concerns**— Expressing the business logic in a dedicated language makes it easier to keep the business-related code isolated from the infrastructural part of the application. The result is code that's easier to maintain.

Conversely, introducing a DSL into your code base can have a few disadvantages:

- **Difficulty of DSL design**— It's hard to capture domain knowledge in a concise limited language.
- **Development cost**— Adding a DSL to your code base is a long-term investment with a high up-front cost, which could delay your project in its early stages. In addition, the maintenance of the DSL and its evolution add further engineering overhead.
- **Additional indirection layer**— A DSL wraps your domain model in an additional layer that has to be as thin as possible to avoid incurring performance problems.
- **Another language to learn**— Nowadays, developers are used to employing multiple languages. Adding a DSL to your project, however, implicitly implies that you and your team have one more language to learn. Worse, if you decide to have multiple DSLs covering different areas of your business domain, combining them in a seamless way could be hard, because DSLs tend to evolve independently.
- **Hosting-language limitations**— Some general-purpose programming languages (Java is one of them) are known for being verbose and having rigid syntax. These languages make it difficult to design a user-friendly DSL. In fact, DSLs developed on top of a verbose programming language are constrained by the cumbersome syntax and may not be nice to read. The introduction of lambda expression in Java 8 offers a powerful new tool to mitigate this problem.

Other formats available



and negative arguments, deciding whether to isn't easy. Moreover, you have alternatives own DSL. Before investigating which pat-

terns and strategies you could employ to develop a readable easy-to-use DSL in Java 8 and beyond, we quickly explore these alternatives and describe the circumstances under which they could be appropriate solutions.

10.1.2. Different DSL solutions available on the JVM

In this section, you learn the categories of DSLs. You also learn that you have many choices besides Java for implementing DSLs. In later sections, we focus on how to implement DSLs by using Java features.

The most common way to categorize DSLs, introduced by Martin Fowler, is to distinguish between internal and external DSLs. Internal DSLs (also known as embedded DSLs) are implemented on top of the existing hosting language (which could be plain Java code), whereas external DSLs are called stand-alone because they're developed from scratch with a syntax that's independent of the hosting language.

Moreover, the JVM gives you a third possibility that falls between an internal and an external DSL: another general-purpose programming language that also runs on the JVM but is more flexible and expressive than Java, such as Scala or Groovy. We refer to this third alternative as a polyglot DSL.

In the following sections, we look at these three types of DSLs in order.

Internal DSL

Because this book is about Java, when we speak about an internal DSL, we clearly mean a DSL written in Java. Historically, Java hasn't been considered to be a DSL-friendly language because its cumbersome, inflexible syntax makes it difficult to produce an easy-to-read, concise, expressive DSL. This issue has been largely mitigated by the introduction of lambda expressions. As you saw in [chapter 3](#), lambdas are useful for using behavior parameterization in a concise manner. In fact, using lambdas extensively results in a DSL with a more acceptable signal/noise ratio by reducing the verbosity that you get with anonymous inner classes. To demonstrate the signal/noise ratio, try to print a list of `Strings` with Java 7 syntax, but use Java 8's new `forEach` method:

```
List<String> numbers = Arrays.asList("one", "two", "three");
numbers.forEach( new Consumer<String>() {
    @Override
    public void accept( String s ) {
        System.out.println(s);
    }
});
```

Other formats available

 Audiobook 

In this snippet, the part that is bold is carrying the signal of the code. All the remaining code is syntactic noise that provides no additional benefit and (even better) is no longer necessary in Java 8. The anonymous inner class can be replaced by the lambda expression

```
numbers.forEach(s -> System.out.println(s));
```

or even more concisely by a method reference:

```
numbers.forEach(System.out::println);
```

You may be happy to build your DSL with Java when you expect users to be somewhat technically minded. If the Java syntax isn't an issue, choosing to develop your DSL in plain Java has many advantages:

- The effort of learning the patterns and techniques necessary to implement a good Java DSL is modest compared with the effort required to learn a new programming language and the tools normally used to develop an external DSL.
- Your DSL is written in plain Java, so it's compiled with the rest of your code. There's no additional building cost caused by the integration of a second language compiler or of the tool employed to generate the external DSL.
- Your development team won't need to get familiar with a different language or with a potentially unfamiliar and complex external tool.
- The users of your DSL will have all the features normally provided by your favorite Java IDE, such as autocompletion and refactoring facilities. Modern IDEs are improving their support for other popular JVM languages, but still don't have support comparable to what they offer Java developers.
- If you need to implement more than one DSL to cover various parts of your domain or multiple domains, you won't have any problem composing them if they're written in plain Java.

Another possibility is combining DSLs that use the same Java bytecode by combining JVM-based programming languages. We call these DSLs polyglot and describe them in the next section.

Polyglot DSL

Nowadays, probably more than 100 languages run on the JVM. Some of these languages, such as Scala and Groovy, are quite popular, and it isn't

Other formats available



to are skilled in them. Other languages, in-ports of other well-known programming other emerging languages, such as Kotlin on mostly because they claim to have features comparable with those of Scala, but with lower intrinsic complexity and a gentle learning curve. All these languages are younger than Java and have been designed with less-constrained, less-verbose syntax. This

characteristic is important because it helps implement a DSL that has less inherent verbosity due to the programming language in which it's embedded.

Scala in particular has several features, such as currying and implicit conversion, that are convenient in developing a DSL. You get an overview of Scala and how it compares to Java in [chapter 20](#). For now, we want to give you a feeling for what you can do with these features by giving you a small example.

Suppose that you want to build a utility function that repeats the execution of another function, `f`, a given number of times. As a first attempt, you could end up with the following recursive implementation in Scala. (Don't worry about the syntax; the overall idea is what's important.)

```
def times(i: Int, f: => Unit): Unit = {  
  f                                     1  
  if (i > 1) times(i - 1, f)          2  
}
```

- **1 Execute the `f` function.**
- **2 If the counter `i` is positive, decrement it and recursively invoke the `times` function.**

Note that in Scala, invoking this function with large values of `i` won't cause a stack overflow, as would happen in Java, because Scala has the tail call optimization, which means that the recursive invocation to the `times` function won't be added to the stack. You learn more about this topic in [chapters 18](#) and [19](#). You can use this function to execute another function repeatedly (one that prints "Hello World" three times) as follows:

```
times(3, println("Hello World"))
```

If you curry the `times` function, or put its arguments in two groups (we cover currying in detail in [chapter 19](#)),

Other formats available



Audiobook



✕ `> Unit): Unit = {`

`1)(f)`

you can achieve the same result by passing the function to be executed multiple times in curly braces:


```
times(3) {
  println("Hello World")
}
```

Finally, in Scala you can define an implicit conversion from an `Int` to an anonymous class by having only one function that in turn has as argument the function to be repeated. Again, don't worry about the syntax and details. The objective of this example is to give you an idea of what's possible beyond Java.

```
implicit def intToTimes(i: Int) = new {           1
  def times(f: => Unit): Unit = {                 2
    def times(i: Int, f: => Unit): Unit = {       3
      f
      if (i > 1) times(i - 1, f)
    }
    times(i, f)                                   4
  }
}
```

- **1 Defines an implicit conversion from an `Int` to an anonymous class**
- **2 The class has only a `times` function accepting another function `f` as argument.**
- **3 A second `times` function takes two arguments and is defined in the scope of the first one.**
- **4 Invokes the inner `times` function**

In this way the user of your small Scala-embedded DSL can execute a function that prints "Hello World" three times as follows:

```
3 times {
  println("Hello World")
}
```

As you can see, the result has no syntactic noise, and it's easily understandable even by a non-developer. Here, the number 3 is automatically converted by the compiler in an instance of a class that stores the number. The `times` function is invoked with dotless notation, and the function to be repeated.

Other formats available



Obtaining a similar result in Java is impossible, so the advantages of using a more DSL-friendly language are obvious. This choice also has some clear inconveniences, however:

- You have to learn a new programming language or have somebody on your team who's already skilled in it. Because developing a nice DSL in these languages generally requires the use of relatively advanced features, superficial knowledge of the new language normally isn't enough.
- You need to complicate your build process a bit by integrating multiple compilers to build the source written with two or more languages.
- Finally, although the majority of languages running on the JVM claim to be 100 percent Java-compatible, making them interoperate with Java often requires awkward tricks and compromises. Also, this interoperation sometimes causes a performance loss. Scala and Java collections aren't compatible, for example, so when a Scala collection has to be passed to a Java function or vice versa, the original collection has to be converted to one that belongs to the native API of the target language.

External DSL

The third option for adding a DSL to your project is implementing an external one. In this case, you have to design a new language from the ground up, with its own syntax and semantics. You also need to set up a separate infrastructure to parse the new language, analyze the output of the parser, and generate the code to execute your external DSL. This is a lot of work! The skills required to perform these tasks are neither common nor easy to acquire. If you do want to go down this road, ANTLR is a parser generator that's commonly used to help and that goes hand in hand with Java.

Moreover, even designing a coherent programming language from scratch isn't a trivial task. Another common problem is that it's easy for an external DSL to grow out of control and to cover areas and purposes for which it wasn't designed.

The biggest advantage in developing an external DSL is the practically unlimited degree of flexibility that it provides. It's possible for you to design a language that perfectly fits the needs and peculiarities of your domain. If you do a good job, the result is an extremely readable language specifically tailored to describe and solve the problems of your business. The other positive outcome is the clear separation between the infrastructural

Other formats available



Audiobook



the business code written with the external DSL is a double-edged sword, however, because it also creates a clear separation between the DSL and the host language.

In the remainder of this chapter, you learn about patterns and techniques that can help you develop effective modern-Java-based internal DSLs. You

start by exploring how these ideas have been used in the design of the native Java API, especially the API additions in Java 8 and beyond.

10.2. Small DSLs in modern Java APIs

The first APIs to take advantage of the new functional capabilities of Java are the native Java APIs themselves. Before Java 8, the native Java API already had a few interfaces with a single abstract method, but as you saw in [section 10.1](#), their use required the implementation of an anonymous inner class with a bulky syntax. The addition of lambdas and (maybe even more important from a DSL point of view) method references changed the rules of the game, making functional interfaces a cornerstone of Java API design.

The `Comparator` interface in Java 8 has been updated with new methods. You learn in [chapter 13](#) that an interface can include both static method and default methods. For now, the `Comparator` interface serves as a good example of how lambdas improve the reusability and composability of methods in native Java API.

Suppose that you have a list of objects representing people (`Persons`), and you want to sort these objects based on the people's ages. Before lambdas, you had to implement the `Comparator` interface by using an inner class:

```
Collections.sort(persons, new Comparator<Person>() {  
    public int compare(Person p1, Person p2) {  
        return p1.getAge() - p2.getAge();  
    }  
});
```

As you've seen in many other examples in this book, now you can replace the inner class with a more compact lambda expression:

```
Collections.sort(people, (p1, p2) -> p1.getAge() - p2.getAge());
```

This technique greatly increases the signal/noise ratio of your code. Java, however, also has a set of static utility methods that let you create `Comparator` objects in a more readable manner. These static methods

Other formats available



or interface. By statically importing the
method, you can rewrite the preceding sorting

```
Collections.sort(persons, comparing(p -> p.getAge()));
```

Even better, you can replace the lambda with a method reference:

```
Collections.sort(persons, comparing(Person::getAge));
```

The benefit of this approach can be pushed even further. If you want to sort the people by age, but in reverse order, you can exploit the instance method `reverse` (also added in Java 8):

```
Collections.sort(persons, comparing(Person::getAge).reverse());
```

Moreover, if you want the people of the same age to be sorted alphabetically, you can compose that `Comparator` with one that performs the comparison on the names:

```
Collections.sort(persons, comparing(Person::getAge)
    .thenComparing(Person::getName));
```

Finally, you could use the new `sort` method added on the `List` interface to tidy things further:

```
persons.sort(comparing(Person::getAge)
    .thenComparing(Person::getName));
```

This small API is a minimal DSL for the domain of collection sorting. Despite its limited scope, this DSL already shows you how a well-designed use of lambdas and method reference can improve the readability, reusability, and composability of your code.

In the next section, we explore a richer and more widely used Java 8 class in which the readability improvement is even more evident: the `Stream` API.

10.2.1. The `Stream` API seen as a DSL to manipulate collections

The `Stream` interface is a great example of a small internal DSL introduced into the native Java API. In fact, a `Stream` can be seen as a compact

Other formats available



ports, transforms, groups, and manipulates data. For example, you could use a `Stream` to filter the lines of a log file that you're required to read a log file starting with the word "ERROR" in a `List<String>`. You could perform this task in an imperative style, as shown in the following listing.

Listing 10.1. Reading the error lines in a log file in imperative style

```

List<String> errors = new ArrayList<>();
int errorCount = 0;
BufferedReader bufferedReader
    = new BufferedReader(new FileReader(fileName));
String line = bufferedReader.readLine();
while (errorCount < 40 && line != null) {
    if (line.startsWith("ERROR")) {
        errors.add(line);
        errorCount++;
    }
    line = bufferedReader.readLine();
}

```

Here, we omitted the error-handling part of the code for brevity. Despite this fact, the code is excessively verbose, and its intention isn't immediately evident. The other aspect that harms both readability and maintainability is the lack of a clear separation of concerns. In fact, the code with the same responsibility is scattered across multiple statements. The code used to read the file line by line, for example, is located in three places:

- Where the `FileReader` is created
- The second condition of the `while` loop, which checks whether the file has terminated
- The end of the `while` loop itself that reads the next line in the file

Similarly, the code that limits the number of lines collected in the list to the first 40 results is scattered across three statements:

- The one initializing the variable `errorCount`
- The first condition of the `while` loop
- The statement incrementing the counter when a line starting with "ERROR" is found in the log

Achieving the same result in a more functional style through the `Stream` interface is much easier and results in far more compact code, as shown in [listing 10.2](#).

Listing 10.2. Reading the error lines in a log file in functional style

Other formats available



Audiobook



```

Files.lines(Paths.get(fileName))
    .filter(line -> line.startsWith("ERROR"))
    .limit(40)
    .collect(toList());

```

- **1 Opens the file and creates a Stream of Strings where each String corresponds to a line in the file.**

- **2 Filters the line starting with “ERROR”.**
- **3 Limits the result to the first 40 lines.**
- **4 Collects the resulting Strings in a List.**

`Files.lines` is a static utility method that returns a `Stream<String>` where each `String` represents a line in the file to be parsed. That part of the code is the only part that has to read the file line by line. In the same way, the statement `limit(40)` is enough to limit the number of collected error lines to the first 40. Can you imagine something more obviously readable?

The fluent style of the `Stream` API is another interesting aspect that’s typical of a well-designed DSL. All intermediate operations are lazy and return another `Stream` allowing a sequence of operations to be pipelined. The terminal operation is eager and triggers the computation of the result of the whole pipeline.

It’s time to investigate the APIs of another small DSL designed to be used in conjunction with the `collect` method of the `Stream` interface: the `Collectors` API.

10.2.2. Collectors as a DSL to aggregate data

You saw that the `Stream` interface can be viewed as a DSL that manipulates lists of data. Similarly, the `Collector` interface can be viewed as a DSL that performs aggregation on data. In [chapter 6](#), we explored the `Collector` interface and explained how to use it to collect, to group, and to partition the items in a `Stream`. We also investigated the static factory methods provided by the `Collectors` class to conveniently create different flavors of `Collector` objects and combine them. It’s time to review how these methods are designed from a DSL point of view. In particular, as the methods in the `Comparator` interface can be combined to support multifeild sorting, `Collectors` can be combined to achieve multilevel grouping. You can group a list of cars, for example, first by their brand and then by their color as follows:

```
Map<String, Map<Color, List<Car>>> carsByBrandAndColor =
    cars.stream().collect(groupingBy(Car::getBrand,
                                     groupingBy(Car::getColor)));
```

Other formats available



pared with what you did to concatenate two
comparators? you defined the multifeild `Comparator` by composing two
`Comparators` in a fluent way,

```
Comparator<Person> comparator =
    comparing(Person::getAge).thenComparing(Person::getName);
```

whereas the Collectors API allows you to create a multilevel Collector by nesting the Collectors:

```
Collector<? super Car, ?, Map<Brand, Map<Color, List<Car>>>>
    carGroupingCollector =
        groupingBy(Car::getBrand, groupingBy(Car::getColor));
```

Normally, the fluent style is considered to be more readable than the nesting style, especially when the composition involves three or more components. Is this difference in style a curiosity? In fact, it reflects a deliberate design choice caused by the fact that the innermost Collector has to be evaluated first, but logically, it's the last grouping to be performed. In this case, nesting the Collector creations with several static methods instead of fluently concatenating them allows the innermost grouping to be evaluated first but makes it appear to be the last one in the code.

It would be easier (except for the use of generics in the definitions) to implement a GroupingBuilder that delegates to the groupingBy factory method but allows multiple grouping operations to be composed fluently. This next listing shows how.

Listing 10.3. A fluent grouping collectors builder

```
import static java.util.stream.Collectors.groupingBy;

public class GroupingBuilder<T, D, K> {
    private final Collector<? super T, ?, Map<K, D>> collector;

    private GroupingBuilder(Collector<? super T, ?, Map<K, D>> collector) {
        this.collector = collector;
    }

    public Collector<? super T, ?, Map<K, D>> get() {
        return collector;
    }
}
```

Other formats available



Audiobook



GroupingBuilder<T, Map<K, D>, J>

after(Function<? super T, ? extends J> classifier)

GroupingBuilder<>(groupingBy(classifier, collector))

```
public static <T, D, K> GroupingBuilder<T, List<T>, K>
    groupOn(Function<? super T, ? extends K> classifier) {
    return new GroupingBuilder<>(groupingBy(classifier));
}
```



```
    }
}
```

What's the problem with this fluent builder? Trying to use it makes the problem evident:

```
Collector<? super Car, ?, Map<Brand, Map<Color, List<Car>>>>
    carGroupingCollector =
        groupOn(Car::getColor).after(Car::getBrand).get()
```

As you can see, the use of this utility class is counterintuitive because the grouping functions have to be written in reverse order relative to the corresponding nested grouping level. If you try to refactor this fluent builder to fix the ordering issue, you realize that unfortunately, the Java type system won't allow you to do this.

By looking more closely at the native Java API and the reasons behind its design decisions, you've started to learn a few patterns and useful tricks for implementing readable DSLs. In the next section, you continue to investigate techniques for developing effective DSLs.

10.3. Patterns and techniques to create DSLs in Java

A DSL provides a friendly, readable API to work with a particular domain model. For that reason, we start this section by defining a simple domain model; then we discuss the patterns that can be used to create a DSL on top of it.

The sample domain model is made of three things. The first thing is plain Java beans modeling a stock quoted on a given market:

```
public class Stock {

    private String symbol;
    private String market;

    public String getSymbol() {
        return symbol;
    }

    public void setSymbol(String symbol) {
        symbol;
    }

    public String getMarket() {
        return market;
    }

    public void setMarket(String market) {
```

Other formats available



✕ symbol(String symbol) {
symbol;

```

        this.market = market;
    }
}

```

The second thing is a trade to buy or sell a given quantity of a stock at a given price:

```

public class Trade {

    public enum Type { BUY, SELL }
    private Type type;

    private Stock stock;
    private int quantity;
    private double price;

    public Type getType() {
        return type;
    }
    public void setType(Type type) {
        this.type = type;
    }

    public int getQuantity() {
        return quantity;
    }
    public void setQuantity(int quantity) {
        this.quantity = quantity;
    }

    public double getPrice() {
        return price;
    }
    public void setPrice(double price) {
        this.price = price;
    }

    public Stock getStock() {
        return stock;
    }
    public void setStock(Stock stock) {
        this.stock = stock;
    }
}

```

Other formats available



Audiobook



```

    value() {
        y * price;
    }
}

```

```

}

```

The final thing is an order placed by a customer to settle one or more trades:

```
public class Order {

    private String customer;
    private List<Trade> trades = new ArrayList<>();

    public void addTrade(Trade trade) {
        trades.add(trade);
    }

    public String getCustomer() {
        return customer;
    }

    public void setCustomer(String customer) {
        this.customer = customer;
    }

    public double getValue() {
        return trades.stream().mapToDouble(Trade::getValue).sum();
    }
}
```

This domain model is straightforward. It's cumbersome to create objects representing orders, for example. Try to define a simple order that contains two trades for your customer BigBank, as shown in [listing 10.4](#).

Listing 10.4. Creating a stock trading order by using the domain object's API directly

```
Order order = new Order();
order.setCustomer("BigBank");

Trade trade1 = new Trade();
trade1.setType(Trade.Type.BUY);

Stock stock1 = new Stock();
stock1.setSymbol("IBM");
stock1.setMarket("NYSE");

trade1.setStock(stock1);

Trade trade2 = new Trade();
trade2.setType(Trade.Type.BUY);

Stock stock2 = new Stock();
```

Other formats available



```

stock2.setSymbol( "GOOGLE" );
stock2.setMarket( "NASDAQ" );

trade2.setStock(stock2);
trade2.setPrice(375.00);
trade2.setQuantity(50);
order.addTrade(trade2);

```

The verbosity of this code is hardly acceptable; you can't expect a non-developer domain expert to understand and validate it at first glance. What you need is a DSL that reflects the domain model and allows it to be manipulated in a more immediate, intuitive way. You can employ various approaches to achieve this result. In the rest of this section, you learn the pros and cons of these approaches.

10.3.1. Method chaining

The first style of DSL to explore is one of the most common. It allows you to define a trading order with a single chain of method invocations. The following listing shows an example of this type of DSL.

Listing 10.5. Creating a stock trading order with method chaining

```

Order order = forCustomer( "BigBank" )
    .buy( 80 )
    .stock( "IBM" )
    .on( "NYSE" )
    .at( 125.00 )
    .sell( 50 )
    .stock( "GOOGLE" )
    .on( "NASDAQ" )
    .at( 375.00 )
    .end();

```

This code looks like a big improvement, doesn't it? It's very likely that your domain expert will understand this code effortlessly. But how can you implement a DSL to achieve this result? You need a few builders that create the objects of this domain through a fluent API. The top-level builder creates and wraps an order, making it possible to add one or more trades to it, as shown in the next listing.

Other formats available



Audiobook



providing a method-chaining DSL

```

public class MethodChainingOrderBuilder {

    public final Order order = new Order();

    private MethodChainingOrderBuilder(String customer) {

```

```

        order.setCustomer(customer);
    }

    public static MethodChainingOrderBuilder forCustomer(String custom
        return new MethodChainingOrderBuilder(customer);
    }

    public TradeBuilder buy(int quantity) {
        return new TradeBuilder(this, Trade.Type.BUY, quantity);
    }

    public TradeBuilder sell(int quantity) {
        return new TradeBuilder(this, Trade.Type.SELL, quantity);
    }

    public MethodChainingOrderBuilder addTrade(Trade trade) {
        order.addTrade(trade);
        return this;
    }

    public Order end() {
        return order;
    }
}

```

- **1 The order wrapped by this builder**
- **2 A static factory method to create a builder of an order placed by a given customer**
- **3 Creates a TradeBuilder to build a trade to buy a stock**
- **4 Creates a TradeBuilder to build a trade to sell a stock**
- **5 Adds a trade to the order**
- **6 Returns the order builder itself, allowing you to fluently create and add further trades**
- **7 Terminates the building of the order and returns it**

The buy() and sell() methods of the order builder create and return another builder that builds a trade and adds it to the order itself:

```

public class TradeBuilder {
    private final MethodChainingOrderBuilder builder;
    public final Trade trade = new Trade();

    public TradeBuilder(MethodChainingOrderBuilder builder,
        Trade.Type type, int quantity) {
        this.builder = builder;
        trade.setType( type );
        trade.setQuantity( quantity );
    }

    public StockBuilder stock(String symbol) {

```

Other formats available



Audiobook



```

        return new StockBuilder(builder, trade, symbol);
    }
}

```

The only public method of the `TradeBuilder` is used to create a further builder, which then builds an instance of the `Stock` class:

```

public class StockBuilder {
    private final MethodChainingOrderBuilder builder;
    private final Trade trade;
    private final Stock stock = new Stock();

    private StockBuilder(MethodChainingOrderBuilder builder,
                        Trade trade, String symbol) {
        this.builder = builder;
        this.trade = trade;
        stock.setSymbol(symbol);
    }

    public TradeBuilderWithStock on(String market) {
        stock.setMarket(market);
        trade.setStock(stock);
        return new TradeBuilderWithStock(builder, trade);
    }
}

```

The `StockBuilder` has a single method, `on()`, that specifies the market for the stock, adds the stock to the trade, and returns one last builder:

```

public class TradeBuilderWithStock {
    private final MethodChainingOrderBuilder builder;
    private final Trade trade;

    public TradeBuilderWithStock(MethodChainingOrderBuilder builder,
                                Trade trade) {
        this.builder = builder;
        this.trade = trade;
    }

    public MethodChainingOrderBuilder at(double price) {
        trade.setPrice(price);
        .addTrade(trade);
    }
}

```

Other formats available



Audiobook



This one public method of `TradeBuilderWithStock` sets the unit price of the traded stock and returns the original order builder. As you've seen, this method allows you to fluently add other trades to the order until the

end method of the `MethodChaining-OrderBuilder` is called. The choice of having multiple builder classes—and in particular, two different trade builders—is made to force the user of this DSL to call the methods of its fluent API in a predetermined sequence, ensuring that a trade has been configured correctly before the user starts creating the next one. The other advantage of this approach is that the parameters used to set an order up are scoped inside the builder. This approach minimizes the use of static methods and allows the names of the methods to act as named arguments, thus further improving the readability of this style of DSL. Finally, the fluent DSL resulting from this technique has the least syntactic noise possible.

Unfortunately, the main issue in method chaining is the verbosity required to implement the builders. A lot of glue code is necessary to mix the top-level builders with the lower-level ones. Another evident disadvantage is the fact that you have no way to enforce the indentation convention that you used to underline the nesting hierarchy of the objects in your domain.

In the next section, you investigate a second DSL pattern that has quite different characteristics.

10.3.2. Using nested functions

The *nested function* DSL pattern takes its name from the fact that it populates the domain model by using functions that are nested within other functions. The following listing illustrates the DSL style resulting from this approach.

Listing 10.7. Creating a stock-trading order with nested functions

```
Order order = order("BigBank",
    buy(80,
        stock("IBM", on("NYSE")),
        at(125.00)),
    sell(50,
        stock("GOOGLE", on("NASDAQ")),
        at(375.00))
);
```

Other formats available



Audiobook



that this DSL style is far more compact than
[10.3.1.](#)

The `NestedFunctionOrderBuilder` in the following listing shows that it's possible to provide an API with this DSL style to your users. (In this listing, we implicitly assume that all its static methods are imported.)

Listing 10.8. An order builder providing a nested-function DSL

```
public class NestedFunctionOrderBuilder {

    public static Order order(String customer, Trade... trades) {
        Order order = new Order();
        order.setCustomer(customer);
        Stream.of(trades).forEach(order::addTrade);
        return order;
    }

    public static Trade buy(int quantity, Stock stock, double price) {
        return buildTrade(quantity, stock, price, Trade.Type.BUY);
    }

    public static Trade sell(int quantity, Stock stock, double price) {
        return buildTrade(quantity, stock, price, Trade.Type.SELL);
    }

    private static Trade buildTrade(int quantity, Stock stock, double price,
                                     Trade.Type buy) {
        Trade trade = new Trade();
        trade.setQuantity(quantity);
        trade.setType(buy);
        trade.setStock(stock);
        trade.setPrice(price);
        return trade;
    }

    public static double at(double price) {
        return price;
    }

    public static Stock stock(String symbol, String market) {
        Stock stock = new Stock();
        stock.setSymbol(symbol);
        stock.setMarket(market);
        return stock;
    }

    public static String on(String market) {
        return market;
    }
}
```

Other formats available



Audiobook



- 3 Creates a trade to buy a stock
- 4 Creates a trade to sell a stock
- 5 A dummy method to define the unit price of the traded stock
- 6 Creates the traded stock

- 7 A dummy method to define the market where the stock is traded

The other advantage of this technique compared with method chaining is that the hierarchy structure of your domain objects (an order contains one or more trades, and each trade refers to a single stock in the example) is visible by the way in which the different functions are nested.

Unfortunately, this pattern also has some issues. You may have noticed that the resulting DSL requires a lot of parentheses. Moreover, the list of arguments that have to be passed to the static methods is rigidly predetermined. If the objects of your domain have some optional fields, you need to implement different overloaded versions of those methods, which allows you to omit the missing parameters. Finally, the meanings of the different arguments are defined by their positions rather than their names. You can mitigate this last problem by introducing a few dummy methods, as you did with the `at()` and `on()` methods in your `NestedFunctionOrderBuilder`, the only purpose of which is to clarify the role of an argument.

The two DSL patterns we've shown you so far don't require the use of lambda expressions. In the next section, we illustrate a third technique that leverages the functional capabilities introduced by Java 8.

10.3.3. Function sequencing with lambda expressions

The next DSL pattern employs a sequence of functions defined with lambda expressions. Implementing a DSL in this style on top of your usual stock-trading domain model allows you to define an order, as shown in [listing 10.9](#).

Listing 10.9. Creating a stock-trading order with function sequencing

```
Order order = order( o -> {
    o.forCustomer( "BigBank" );
    o.buy( t -> {
        t.quantity( 80 );
        t.price( 125.00 );
        t.stock( s -> {
            s.symbol( "IBM" );
            s.market( "NYSE" );
        } );
    } );
    t.price( 375.00 );
    t.stock( s -> {
        s.symbol( "GOOGLE" );
        s.market( "NASDAQ" );
    } );
} );
```

Other formats available



Audiobook



```
    });  
  } );
```

To implement this approach, you need to develop several builders that accept lambda expressions and to populate the domain model by executing them. These builders keep the intermediate state of the objects to be created the way you did in the DSL implementation by using method chaining. As you did in the method-chaining pattern, you have a top-level builder to create the order, but this time, the builder takes Consumer objects as parameters so that the user of the DSL can use lambda expressions to implement them. The next listing shows the code required to implement this approach.

Listing 10.10. An order builder providing a function-sequencing DSL

```
public class LambdaOrderBuilder {  
  
    private Order order = new Order();  
  
    public static Order order(Consumer<LambdaOrderBuilder> consumer) {  
        LambdaOrderBuilder builder = new LambdaOrderBuilder();  
        consumer.accept(builder);  
        return builder.order;  
    }  
  
    public void forCustomer(String customer) {  
        order.setCustomer(customer);  
    }  
  
    public void buy(Consumer<TradeBuilder> consumer) {  
        trade(consumer, Trade.Type.BUY);  
    }  
  
    public void sell(Consumer<TradeBuilder> consumer) {  
        trade(consumer, Trade.Type.SELL);  
    }  
  
    private void trade(Consumer<TradeBuilder> consumer, Trade.Type type)  
        TradeBuilder builder = new TradeBuilder();  
        builder.trade.setType(type);  
        consumer.accept(builder);  
        order.addTrade(builder.trade);  
    }
```

Other formats available



Audiobook



- 1 The order wrapped by this builder
- 2 Executes the lambda expression passed to the order builder
- 3 Returns the order populated by executing the Consumer of the OrderBuilder

- 4 Sets the customer who placed the order
- 5 Consumes a TradeBuilder to create a trade to buy a stock
- 6 Consumes a TradeBuilder to create a trade to sell a stock
- 7 Executes the lambda expression passed to the TradeBuilder
- 8 Adds to the order the trade populated by executing the Consumer of the TradeBuilder

The buy() and sell() methods of the order builder accept two lambda expressions that are Consumer<TradeBuilder>. When executed, these methods populate a buying or selling trade, as follows:

```
public class TradeBuilder {
    private Trade trade = new Trade();

    public void quantity(int quantity) {
        trade.setQuantity( quantity );
    }

    public void price(double price) {
        trade.setPrice( price );
    }

    public void stock(Consumer<StockBuilder> consumer) {
        StockBuilder builder = new StockBuilder();
        consumer.accept(builder);
        trade.setStock(builder.stock);
    }
}
```

Finally, the TradeBuilder accepts the Consumer of a third builder that's intended to define the traded stock:

```
public class StockBuilder {
    private Stock stock = new Stock();

    public void symbol(String symbol) {
        stock.setSymbol( symbol );
    }

    public void market(String market) {
        stock.setMarket( market );
    }
}
```

Other formats available



This pattern has the merit of combining two positive characteristics of the two previous DSL styles. Like the method-chaining pattern it allows to define the trading order in a fluent way. In addition, similarly to the nested-

function style, it preserves the hierarchy structure of our domain objects in the nesting level of the different lambda expressions.

Unfortunately, this approach requires a lot of setup code, and using the DSL itself is affected by the noise of the Java 8 lambda-expression syntax.

Choosing among these three DSL styles is mainly a matter of taste. It also requires some experience to find the best fit for the domain model for which you want to create a domain language. Moreover, it's possible to combine two or more of these styles in a single DSL, as you see in the next section.

10.3.4. Putting it all together

As you've seen so far, all three DSL patterns have pros and cons, but nothing prevents you from using them together within a single DSL. You could end up developing a DSL through which you could define your stock-trading order as shown in the following listing.

Listing 10.11. Creating a stock-trading order by using multiple DSL patterns

```
Order order =  
    forCustomer( "BigBank",  
        buy( t -> t.quantity( 80 )  
            .stock( "IBM" )  
            .on( "NYSE" )  
            .at( 125.00 ) ),  
        sell( t -> t.quantity( 50 )  
            .stock( "GOOGLE" )  
            .on( "NASDAQ" )  
            .at( 125.00 ) ) );
```

- **1 Nested function to specify attributes of the top-level order**
- **2 Lambda expression to create a single trade**
- **3 Method chaining in the body of the lambda expression that populates the trade object**

In this example, the nested-function pattern is combined with the lambda approach. Each trade is created by a Consumer of a TradeBuilder that's

Other formats available



Audiobook



expression, as shown in the next listing.

er providing a DSL that mixes multiple

```
public class MixedBuilder {
```

```
    public static Order forCustomer(String customer,
```

```

        TradeBuilder... builders) {
    Order order = new Order();
    order.setCustomer(customer);
    Stream.of(builders).forEach(b -> order.addTrade(b.trade));
    return order;
}

public static TradeBuilder buy(Consumer<TradeBuilder> consumer) {
    return buildTrade(consumer, Trade.Type.BUY);
}

public static TradeBuilder sell(Consumer<TradeBuilder> consumer) {
    return buildTrade(consumer, Trade.Type.SELL);
}

private static TradeBuilder buildTrade(Consumer<TradeBuilder> consumer,
                                       Trade.Type buy) {
    TradeBuilder builder = new TradeBuilder();
    builder.trade.setType(buy);
    consumer.accept(builder);
    return builder;
}
}

```

Finally, the helper class `TradeBuilder` and the `StockBuilder` that it uses internally (implementation shown immediately after this paragraph) provide a fluent API implementing the method-chaining pattern. After you make this choice, you can write the body of the lambda expression through which the trade will be populated in the most compact way possible:

```

public class TradeBuilder {
    private Trade trade = new Trade();

    public TradeBuilder quantity(int quantity) {
        trade.setQuantity(quantity);
        return this;
    }

    public TradeBuilder at(double price) {
        trade.setPrice(price);
        return this;
    }

    public TradeBuilder for stock(String symbol) {
        stockBuilder(this, trade, symbol);
    }
}

public class StockBuilder {
    private final TradeBuilder builder;

```

Other formats available



```

private final Trade trade;
private final Stock stock = new Stock();

private StockBuilder(TradeBuilder builder, Trade trade, String symbol) {
    this.builder = builder;
    this.trade = trade;
    stock.setSymbol(symbol);
}

public TradeBuilder on(String market) {
    stock.setMarket(market);
    trade.setStock(stock);
    return builder;
}
}

```

Listing 10.12 is an example of how the three DSL patterns discussed in this chapter can be combined to achieve a readable DSL. Doing so allows you to take advantage of the pros of the various DSL styles, but this technique has a minor drawback: the resulting DSL appears to be less uniform than one that uses a single technique, so users of this DSL probably will need more time to learn it.

So far, you've used lambda expressions, but, as the `Comparator` and `Stream` APIs show, using method references can further improve the readability of many DSLs. We demonstrate this fact in the next section through a practical example of using method references in the stock-trading domain model.

10.3.5. Using method references in a DSL

In this section, you try to add another simple feature to your stock-trading domain model. This feature calculates the final value of an order after adding zero or more of the following taxes to the order's net value, as shown in the next listing.

Listing 10.13. The taxes that can be applied to the order's net value

```

public class Tax {
    public static double regional(double value) {
        return value * 1.1;
    }

    public static double general(double value) {
        return value * 1.3;
    }

    public static double surcharge(double value) {
        return value * 1.05;
    }
}

```

Other formats available



Audiobook



```
}  
}
```

The simplest way to implement such a tax calculator is to use a static method that accepts the order plus one Boolean flag for each tax that could be applied ([listing 10.14](#)).

Listing 10.14. Applying taxes to the order's net value with a set of Boolean flags

```
public static double calculate(Order order, boolean useRegional,  
                               boolean useGeneral, boolean useSurcharg  
    double value = order.getValue();  
    if (useRegional) value = Tax.regional(value);  
    if (useGeneral) value = Tax.general(value);  
    if (useSurcharge) value = Tax.surcharge(value);  
    return value;  
}
```

This way, it's possible to calculate the final value of an order after applying the regional tax and the surcharge, but not the general tax, as follows:

```
double value = calculate(order, true, false, true);
```

The readability problem of this implementation is evident: it's difficult to remember the right sequence of Boolean variables and to understand which taxes have been applied and which haven't. The canonical way to fix this issue is to implement a `TaxCalculator` that provides a minimal DSL to fluently set the Boolean flags one by one, as shown the next listing.

Listing 10.15. A tax calculator that fluently defines the taxes to be applied

```
public class TaxCalculator {  
    private boolean useRegional;  
    private boolean useGeneral;  
    private boolean useSurcharge;  
  
    public TaxCalculator withTaxRegional() {  
        useRegional = true;  
  
    public TaxCalculator withTaxGeneral() {  
        useGeneral = true;  
        return this;  
    }  
}
```

Other formats available




```

    public TaxCalculator withTaxSurcharge() {
        useSurcharge = true;
        return this;
    }

    public double calculate(Order order) {
        return calculate(order, useRegional, useGeneral, useSurcharge)
    }
}

```

Using this `TaxCalculator` makes clear that you want to apply the regional tax and the surcharge to the net value of the order:

```

double value = new TaxCalculator().withTaxRegional()
                                   .withTaxSurcharge()
                                   .calculate(order);

```

The main issue with this solution is its verbosity. It doesn't scale because you need a Boolean field and a method for each tax in your domain. By using the functional capabilities of Java, you can achieve the same result in terms of readability in a far more compact and flexible way. To see how, refactor your `TaxCalculator` as shown in this next listing.

Listing 10.16. A tax calculator that fluently combines the tax functions to be applied

```

public class TaxCalculator {
    public DoubleUnaryOperator taxFunction = d -> d; 1

    public TaxCalculator with(DoubleUnaryOperator f) {
        taxFunction = taxFunction.andThen(f); 2
        return this; 3
    }

    public double calculate(Order order) {
        return taxFunction.applyAsDouble(order.getValue()); 4
    }
}

```

Other formats available



Audiobook



all the taxes to be applied to the order's

calculating function, composing the current

one with the one passed as argument

- 3 Returns this, allowing further tax functions to be concatenated fluently

- **4 Calculates the final order's value by applying the tax-calculating function to the order's net value**

With this solution, you need only one field: the function that, when applied to the order's net value, adds in one shot all the taxes configured through the `TaxCalculator` class. The starting value of this function is the identity function. At this point, no tax has been added yet, so the order's final value is the same as the net value. When a new tax is added through the `with()` method, this tax is composed with the current tax-calculating function, thus encompassing all the added taxes in a single function. Finally, when an order is passed to the `calculate()` method, the tax-calculating function resulting from the composition of the various configured taxes is applied to the order's net value. This refactored `TaxCalculator` can be used as follows:

```
double value = new TaxCalculator().with(Tax::regional)
                                   .with(Tax::surcharge)
                                   .calculate(order);
```

This solution uses method references, is easy to read, and gives succinct code. It's also flexible in that if and when a new tax function is added to the `Tax` class, you can use it immediately with your functional `TaxCalculator` without modification.

Now that we've discussed the various techniques that can be used to implement a DSL in Java 8 and beyond, it's interesting to investigate how these strategies have been used in widely adopted Java tools and frameworks.

10.4. Real World Java 8 DSL

In [section 10.3](#), you learned three useful patterns for developing DSLs in Java, together with their pros and cons. [Table 10.1](#) summarizes what we've discussed so far.

Table 10.1. DSLs patterns with their pros and cons

Pattern	Pros	Cons
Other formats available  Audiobook  chaining	• Method names that act as key-word arguments • Works well with optional parameters	• Verbose implementation

Pattern name	Pros	Cons
	• Possible to enforce the DSL user to call methods in a pre-determined order • Minimal or no use of static methods • Lowest possible syntactic noise	• Glue code to bind the builders • Hierarchy of domain objects defined only by indentation convention
Nested functions	• Lower implementation verbosity • Domain objects hierarchy echoed by function nesting	• Heavy use of static methods • Arguments defined by position rather than name • Method overloading required for optional parameters
Function sequencing with lambdas	• Works well with optional parameters • Minimal or no use of static methods • Hierarchy of domain objects echoed by lambdas nesting • No glue code for builders	• Verbose implementation • More syntactic noise from lambda expressions in the DSL

Other formats available



Audiobook



It's time to consolidate what you've learned so far by analyzing how these patterns are employed in three well-known Java libraries: an SQL mapping tool, a behavior-driven development framework, and a tool that implements Enterprise Integration Patterns.

10.4.1. jOOQ

SQL is one of the most common and widely used DSLs. For this reason, it shouldn't be surprising that there's a Java library providing a nice DSL to write and execute SQL queries. jOOQ is an internal DSL that implements SQL as a type-safe embedded language directly in Java. A source-code generator reverse-engineers the database schema, which allows the Java compiler to type-check complex SQL statements. The product of this reverse-engineering process generates information with which you can navigate your database schema. As a simple example, the following SQL query

```
SELECT * FROM BOOK
WHERE BOOK.PUBLISHED_IN = 2016
ORDER BY BOOK.TITLE
```

can be written using the jOOQ DSL like this:

```
create.selectFrom(BOOK)
    .where(BOOK.PUBLISHED_IN.eq(2016))
    .orderBy(BOOK.TITLE)
```

Another nice feature of the jOOQ DSL is the possibility of using it in combination with the `Stream` API. This feature allows you to manipulate in memory, with a single fluent statement, the data resulting from the execution of the SQL query, as shown in the next listing.

Listing 10.17. Selecting books from a database by using the jOOQ DSL

```
Class.forName("org.h2.Driver");
try (Connection c =
    getConnection("jdbc:h2:~/sql-goodies-with-mapping", "sa", ""))
    DSL.using(c)
        .select(BOOK.AUTHOR, BOOK.TITLE)
        .where(BOOK.PUBLISHED_IN.eq(2016))
        .orderBy(BOOK.TITLE)
        .fetch()
        .stream()
        .collect(groupingBy(
            (BOOK.AUTHOR),
            new,
            getValue(BOOK.TITLE), toList()))
        .forEach((author, titles) ->
            System.out.println(author + " is author of " + titles));
}
```

Other formats available



Audiobook



- **1 Creates the connection to the SQL database**
- **2 Starts the jOOQ SQL statement, using the just-created database connection**
- **3 Defines the SQL statement through the jOOQ DSL**
- **4 Fetches the data from the database; jOOQ statement ends here**
- **5 Starts manipulating data fetched from database with Stream API**
- **6 Groups the books by author**
- **7 Prints the authors' names together with the books they wrote**

It's evident that the main DSL pattern chosen to implement the jOOQ DSL is method-chaining. In fact, various characteristics of this pattern (allowing optional parameters and requiring certain methods to be called only in a predetermined sequence) are essential to mimic the syntax of a well-formed SQL query. These features, together with its lower syntactic noise, make the method-chaining pattern a good fit for jOOQ's needs.

10.4.2. Cucumber

Behavior-driven development (BDD) is an extension of test-driven development that uses a simple domain-specific scripting language made of structured statements that describe various business scenarios.

Cucumber, like other BDD frameworks, translates these statements into executable test cases. As a result, the scripts resulting from the application of this development technique can be used both as runnable tests and as acceptance criteria for a given business feature. BDD also focuses the development effort on the delivery of prioritized, verifiable business value and bridges the gap between domain experts and programmers by making them share a business vocabulary.

These abstract concepts can be clarified by a practical example that uses Cucumber, a BDD tool that enables developers to write business scenarios in plain English. Use Cucumber's scripting language as follows to define a simple business scenario:

```
Feature: Buy stock
  Scenario: Buy 10 IBM stocks
    Given the price of a "IBM" stock is 125$
    When I buy 10 "IBM"
    Then the order value should be 1250$
```

Other formats available



divided into three parts: the definition of
al calls to the domain objects under test,
and (When) the assertions checking the outcome of the test case (Then).

The script that defines the test scenario is written with an external DSL that has a limited number of keywords and lets you write sentences in a

free format. These sentences are matched through regular expressions that capture the variables of the test case and pass them as arguments to the methods that implement the test itself. Using the stock-trading domain model from the beginning of [section 10.3](#), it's possible to develop a Cucumber test case that checks whether the value of a stock-trading order is calculated correctly, as shown in the next listing.

Listing 10.18. Implementing a test scenario by using Cucumber annotations

```
public class BuyStocksSteps {
    private Map<String, Integer> stockUnitPrices = new HashMap<>();
    private Order order = new Order();

    @Given("^the price of a \"(.*)\" stock is (\\d+)\\$\\$")           1
    public void setUnitPrice(String stockName, int unitPrice) {
        stockUnitPrices.put(stockName, unitPrice);                 2
    }

    @When("^I buy (\\d+) \"(.*)\"")                                   3
    public void buyStocks(int quantity, String stockName) {
        Trade trade = new Trade();                                  4
        trade.setType(Trade.Type.BUY);

        Stock stock = new Stock();
        stock.setSymbol(stockName);

        trade.setStock(stock);
        trade.setPrice(stockUnitPrices.get(stockName));
        trade.setQuantity(quantity);
        order.addTrade(trade);
    }

    @Then("^the order value should be (\\d+)\\$\\$")                 5
    public void checkOrderValue(int expectedValue) {
        assertEquals(expectedValue, order.getValue());            6
    }
}
```

- 1 Defines the unit price of a stock as a prerequisite of this scenario
- 2 Stores the stock unit price
- 3 Defines the actions to be taken on the domain model under test

Other formats available



Audiobook



Model accordingly

Scenario outcome

is

The introduction of lambda expressions in Java 8 allowed Cucumber to develop an alternative syntax that eliminated annotations by using two-argument methods: the regular expression previously contained in the

annotation value and the lambda implementing the test method. When you use this second type of notation, you can rewrite the test scenario like this:

```
public class BuyStocksSteps implements cucumber.api.java8.En {
    private Map<String, Integer> stockUnitPrices = new HashMap<>();
    private Order order = new Order();
    public BuyStocksSteps() {
        Given("^the price of a \"(.*)\" stock is (\\d+)\\$",
            (String stockName, int unitPrice) -> {
                stockUnitValues.put(stockName, unitPrice);
            });
        // ... When and Then lambdas omitted for brevity
    }
}
```

This alternative syntax has the obvious advantage of being compact. In particular, replacing the test methods with anonymous lambdas eliminates the burden of finding meaningful method names (which rarely adds anything to readability in a test scenario).

Cucumber's DSL is extremely simple, but it demonstrates how to effectively combine an external DSL with an internal one and (once again) shows that lambdas allow you to write more compact, readable code.

10.4.3. Spring Integration

Spring Integration extends the dependency-injection-based Spring programming model to support the well-known Enterprise Integration Patterns.^[1] Spring Integration's primary goals are to provide a simple model to implement complex enterprise integration solutions and to promote the adoption of an asynchronous, message-driven architecture.

¹

For more details see the book: "Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions" (Addison-Wesley) Gregor Hohpe and Bobby Woolf, 2004.

Spring Integration enables lightweight remoting, messaging, and sched-

Other formats available



ulations. These features are also available
's more than syntactic sugar built on top of
uration files.

Spring Integration implements all the most common patterns necessary for message-based applications, such as channels, endpoints, pollers, and channel interceptors. Endpoints are expressed as verbs in the DSL to improve readability, and integration processes are constructed by compos-

ing these endpoints into one or more message flows. The next listing shows how Spring Integration works with a simple but complete example.

Listing 10.19. Configuring a Spring Integration flow by using the Spring Integration DSL

```
@Configuration
@EnableIntegration
public class MyConfiguration {

    @Bean
    public MessageSource<?> integerMessageSource() {
        MethodInvokingMessageSource source =
            new MethodInvokingMessageSource();
        source.setObject(new AtomicInteger());
        source.setMethodName("getAndIncrement");
        return source;
    }

    @Bean
    public DirectChannel inputChannel() {
        return new DirectChannel();
    }

    @Bean
    public IntegrationFlow myFlow() {
        return IntegrationFlows
            .from(this.integerMessageSource(),
                c -> c.poller(Pollers.fixedRate(10)))
            .channel(this.inputChannel())
            .filter((Integer p) -> p % 2 == 0)
            .transform(Object::toString)
            .channel(MessageChannels.queue("queueChannel"))
            .get();
    }
}
```

- **1** Creates a new Message-Source that at each invocation increments an AtomicInteger
- **2** The channel conveying the data arriving from the MessageSource

Other formats available



Audiobook



✕ IntegrationFlow through a builder following the MessageSource pattern

used MessageSource as the source for this

IntegrationFlow

- **5** Polls the MessageSource to dequeue the data it conveys
- **6** Filters only the even numbers

- 7 Converts the Integers retrieved from the MessageSource into Strings
- 8 Sets channel queueChannel as output for this IntegrationFlow
- 9 Terminates the building of the IntegrationFlow and returns it

Here, the method `myFlow()` builds an `IntegrationFlow` by using the Spring Integration DSL. It uses the fluent builder provided by the `IntegrationFlows` class, which implements the method-chaining pattern. In this case, the resulting flow polls a `MessageSource` at a fixed rate, providing a sequence of `Integers`; filters the even ones and converts them to `Strings`, and finally sends the result to an output channel in a style that's similar to the native Java 8 `Stream` API. This API allows a message to be sent to any component within the flow if you know its `inputChannel` name. If the flow starts with a direct channel, not a `MessageSource`, it's possible to define the `IntegrationFlow` with a lambda expression as follows:

```
@Bean
public IntegrationFlow myFlow() {
    return flow -> flow.filter((Integer p) -> p % 2 == 0)
        .transform(Object::toString)
        .handle(System.out::println);
}
```

As you see, the most widely used pattern in Spring Integration DSL is method chaining. This pattern fits well with the main purpose of the `IntegrationFlow` builder: creating a flow of message-passing and data transformations. As shown in this last example, however, it also uses function sequencing with lambda expressions for the top-level object to be built (and in some cases also for inner, more-complex method arguments).

Summary

- The main purpose of a DSL is to fill the gap between developers and domain experts. It's rare for the person who writes the code that implements the business logic of an application to also have deep knowledge in the business field in which the program will be used. Writing

Other formats available



Audiobook



uage that non-developers can understand
s into programmers, but it does allow them
ic.

DSLs are *internal* (implemented in the same language used to develop the application in which the DSL will be used) and *external* (using a different language designed ad hoc). Internal DSLs require less development effort but have a syntax con-

strained by the hosting language. External DSLs offer a higher degree of flexibility but are harder to implement.

- It's possible to develop a polyglot DSL by using another programming language already available on the JVM, such as Scala or Groovy. These languages are often more flexible and concise than Java. Integrating them with Java requires a more-complex building process, however, and their interoperability with Java can be far from seamless.
- Due to its verbosity and rigid syntax, Java isn't the ideal programming language to use to develop internal DSLs, but the introduction of lambda expressions and method references in Java 8 hugely improved this situation.
- Modern Java already provides small DSLs in its native API. These DSLs, like the ones in the `Stream` and `Collectors` classes, are useful and convenient, particularly for sorting, filtering, transforming, and grouping collections of data.
- The three main patterns used to implement DSLs in Java are method chaining, nested functions, and function sequencing. Each pattern has pros and cons, but you can combine all three patterns in a single DSL to take advantage of all three techniques.
- Many Java frameworks and libraries allow their features to be used through a DSL. This chapter looked at three of them: jOOQ, an SQL mapping tool; Cucumber, a BDD framework; and Spring Integration, a Spring extension that implements Enterprise Integration Patterns.

Other formats available



Audiobook

